

Comunicación entre Procesos

Sistemas Operativos



Comunicación entre Procesos

- Los procesos poseen espacios de memoria independientes y no pueden acceder directamente a la información de otros procesos.
- *¿Cómo pueden intercambiar información dos o más procesos?*
- *¿Cómo evitar que el proceso B intente leer los datos antes de que el proceso A termine de escribirlos?*

Ejemplos

- Navegador web y proceso de red.
- Editor de texto e impresora.
- Aplicaciones cliente-servidor.



Comunicación entre Procesos

- La Comunicación entre Procesos (Inter Process Communication, IPC) es el conjunto de mecanismos proporcionados por el sistema operativo para permitir el intercambio de información y la coordinación entre procesos.
- Compartir información.
- Sincronizar procesos.
- Construir aplicaciones distribuidas.

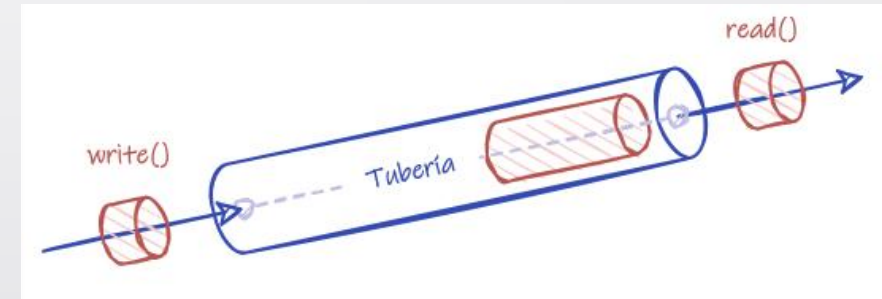


Comunicación entre Procesos

Tipo	Velocidad	Sincronización	Complejidad	Uso recomendado
Pipes	Alta	Media	Baja	Comunicación padre-hijo
Señales	Muy alta	Baja	Baja	Notificaciones y alertas
FIFOs	Media	Media	Baja	Comunicación entre procesos sin relación
Memoria Compartida	Muy alta	Alta	Media-Alta	Alto rendimiento
Colas de Mensajes	Media	Alta	Media	Mensajes estructurados
Semáforos	Alta	Muy alta	Media	Sincronización y exclusión mutua

Tuberías (PIPES) Anónimas

- Son uno de los métodos más sencillos de IPC
- Es un mecanismo de comunicación unidireccional
- Tiene dos extremos: uno de lectura y otro de escritura.
- Se usa principalmente entre procesos padre e hijo (después de un `fork()`).
- Es temporal (desaparece cuando terminan los procesos).



Comunicación entre Procesos

Relación entre procesos	¿Se puede con Pipe Anónimo?	¿Es fácil?	Recomendado
Padre ↔ Hijo	Sí	Muy fácil	★★★★★
Hermano ↔ Hermano	Sí (vía padre)	Medio	★★★
Nieto ↔ Abuelo / Padre	Sí	Difícil	★★
Procesos sin relación	No	Imposible	No usar

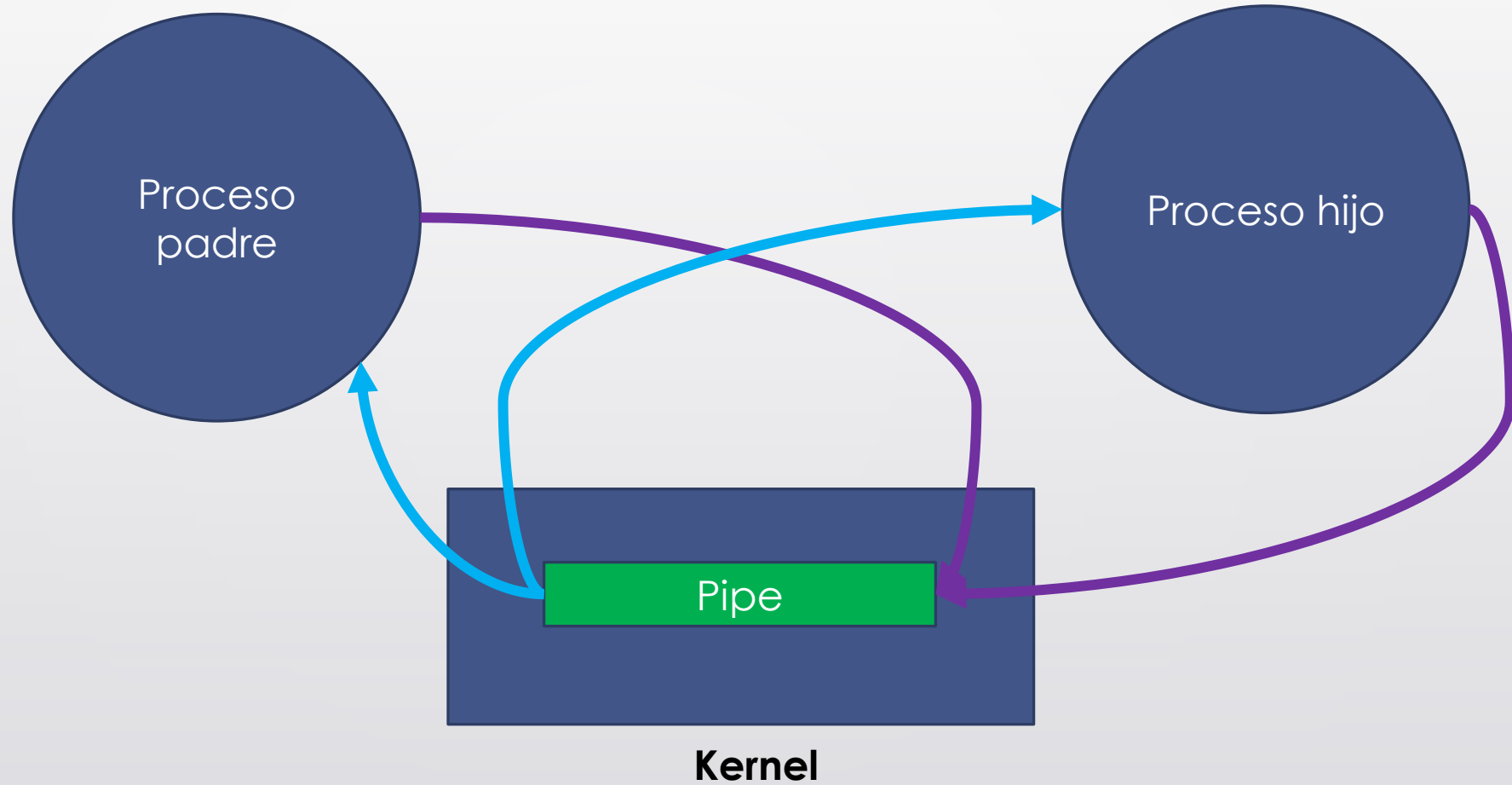
Tuberías (PIPES)

- Se crea mediante la llamada al sistema **pipe(fd[2])**, que devuelve dos descriptors de fichero asociado :
 - Lectura (fd[0])
 - Escritura (fd[1])



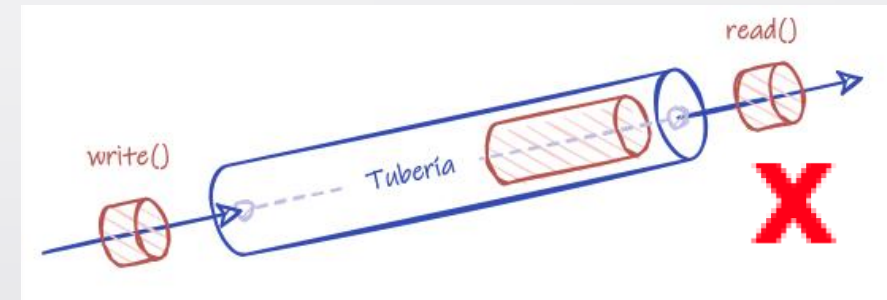
Se lee por uno de ellos y se escribe por el otro

Tuberías (PIPES)



Tuberías (PIPES)

- Cuando se crea un pipe mediante `pipe()`, tanto el proceso padre como el hijo heredan los descriptores de archivo después del `fork()`.
- Ambos procesos pueden utilizar `fd[0]` y `fd[1]`.
- Para evitar comportamientos indeseados, cada proceso debe cerrar el extremo de la tubería que no utilice.



¿Por qué debemos crear el pipe antes del fork bifurcación?

Tuberías (PIPES)

- Garantiza que tanto el proceso padre como el hijo tengan acceso a los mismos descriptores de fichero.
- Si se crean después del fork: Los procesos padre e hijo tendrán tablas de descriptores diferentes.
- No compartirán el mismo pipe, por lo que la comunicación no será posible.



Tuberías (PIPES)

- Para leer y escribir de la tubería se utilizan las llamadas al sistema

- write

- `int write( int fichero, void* buffer, unsigned bytes );`
 - `write(descriptor[1], &buf, sizeof(buf))`

- read

- `int read ( int fichero, void* buffer, unsigned bytes );`
 - `read(descriptor[0], &buf, sizeof(buf))`

Ambas devuelven el número de bytes transferidos



Ejemplo

Codifica lo siguiente en tu SO con arquitectura monolítica


```

#include<stdio.h>
#include<unistd.h>
#include <sys/wait.h>

int factorial(int n){
    int f=1;
    for(int i=1; i<=n; i++){
        f*=i;
    }
    return f;
}

int main(){

    int fd[2];
    pipe(fd);

    pid_t pid= fork();

    if(pid>0){

        close(fd[0]);
        int num=5;
        write(fd[1], &num, sizeof(num));
        close(fd[1]);
        wait(NULL);
    }
    else if(pid==0){
        close(fd[1]);
        int n;
        read(fd[0], &n, sizeof(n));
        printf("El hijo lee del pipe el dato: %d\n", n);
        printf("El factorial de %d es %d\n", n, factorial(n));
        close(fd[0]);
    }

    return 0;
}

```

1

```
#include<stdio.h>
#include<unistd.h>
#include <sys/wait.h>

int factorial(int n){
    int f=1;
    for(int i=1; i<=n; i++){
        f*=i;
    }
    return f;
}
```

2

```
int main(){

    int fd[2];
    pipe(fd);

    pid_t pid= fork();

    if(pid>0){

        close(fd[0]);
        int num=5;
        write(fd[1], &num, sizeof(num));
        close(fd[1]);
        wait(NULL);
    }
    else if(pid==0){
        close(fd[1]);
        int n;
        read(fd[0], &n, sizeof(n));
        printf("El hijo lee del pipe el dato: %d\n", n);
        printf("El factorial de %d es %d\n", n, factorial(n));
        close(fd[0]);
    }

    return 0;
}
```

```

#include<stdio.h>
#include<unistd.h>
#include <sys/wait.h>

int factorial(int n){
    int f=1;
    for(int i=1; i<=n; i++){
        f*=i;
    }
    return f;
}

int main(){

    int fd[2];
    pipe(fd);

    pid_t pid= fork();

    if(pid>0){

        close(fd[0]);
        int num=5;
        write(fd[1], &num, sizeof(num));
        close(fd[1]);
        wait(NULL);
    }
    else if(pid==0){
        close(fd[1]);
        int n;
        read(fd[0], &n, sizeof(n));
        printf("El hijo lee del pipe el dato: %d\n", n);
        printf("El factorial de %d es %d\n", n, factorial(n));
        close(fd[0]);
    }

    return 0;
}

```

```

alumnado@debianS0:~/Documentos/programas$ gcc pipe1.c -o pipe1
alumnado@debianS0:~/Documentos/programas$ ./pipe1
El hijo lee del pipe el dato: 5
El factorial de 5 es 120

```



¿Qué cambios harías para obtener una comunicación bidireccional?

1

```
int main(){

    int fd1[2]; // padre escribe, hijo lee
    int fd2[2]; // hijo escribe, padre lee
    pipe(fd1); //crear los dos pipes
    pipe(fd2);

    pid_t pid= fork();

    if(pid>0){

        close(fd1[0]);
        int num=5;
        write(fd1[1], &num, sizeof(num));
        printf("Padre envia: %d\n", num);
        close(fd1[1]);

        // cambio importante, revisen bien
        close(fd2[1]);
        int resultado;
        read(fd2[0], &resultado, sizeof(resultado));
        printf("Padre recibe del pipe el factorial: %d\n", resultado);
        close(fd2[2]);

        wait(NULL);
    }
```

2

```
    }
    else if(pid==0){
        close(fd1[1]);
        int n;
        read(fd1[0], &n, sizeof(n));
        printf("El hijo lee del pipe el dato: %d\n", n);
        close(fd1[0]);

        // cambio importante, revisen bien
        close(fd2[0]);
        int r= factorial(n);
        write(fd2[1], &r, sizeof(r));
        printf("Hijo envia resultado: %d\n", r);
        close(fd2[1]);
    }
    return 0;
}
```



```

#include<stdio.h>
#include<unistd.h>
#include <sys/wait.h>

int factorial(int n){
    int f=1;
    for(int i=1; i<=n; i++){
        f*=i;
    }
    return f;
}

int main(){

    int fd1[2]; // padre escribe, hijo lee
    int fd2[2]; // hijo escribe, padre lee
    pipe(fd1); //crear los dos pipes
    pipe(fd2);

    pid_t pid= fork();

    if(pid>0){
        close(fd1[0]);
        int num=5;
        write(fd1[1], &num, sizeof(num));
        printf("Padre envia: %d\n", num);
        close(fd1[1]);

        // cambio importante, revisen bien
        close(fd2[1]);
        int resultado;
        read(fd2[0], &resultado, sizeof(resultado));
        printf("Padre recibe del pipe el factorial: %d\n", resultado);
        close(fd2[2]);

        wait(NULL);
    }
    else if(pid==0){
        close(fd1[1]);
        int n;
        read(fd1[0], &n, sizeof(n));
        printf("El hijo lee del pipe el dato: %d\n", n);
        close(fd1[0]);

        // cambio importante, revisen bien
        close(fd2[0]);
        int r= factorial(n);
        write(fd2[1], &r, sizeof(r));
        printf("Hijo envia resultado: %d\n", r);
        close(fd2[1]);
    }
    return 0;
}

```

```

alumnado@debianS0:~/Documentos/programas$ ./pipe2
Padre envia: 5
El hijo lee del pipe el dato: 5
Hijo envia resultado: 120
Padre recibe del pipe el factorial: 120

```

Comunicación entre hermanos

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
```

1

```
int factorial(int n)
{
    int f = 1;

    for(int i = 1; i <= n; i++)
    {
        f *= i;
    }

    return f;
}
```

```
int main()
{
    int fd[2];

    pipe(fd);

    pid_t hijo1 = fork();

    if(hijo1 == 0)
    {
        // Hijo 1 es el que escribe

        close(fd[0]); // No lee

        int num = 5;

        printf("Hijo 1 envia: %d\n", num);

        write(fd[1], &num, sizeof(num));

        close(fd[1]); // Termina de escribir

        return 0;
    }
}
```

2

```
pid_t hijo2 = fork();

if(hijo2 == 0)
{
    // Hijo 2 es el que lee

    close(fd[1]); // No escribe

    int n;

    read(fd[0], &n, sizeof(n));

    printf("Hijo 2 recibe: %d\n", n);
    printf("Factorial de %d = %d\n", n, factorial(n));

    close(fd[0]); // Termina de leer

    return 0;
}

// Padre cierra los descriptores

close(fd[0]);
close(fd[1]);

wait(NULL);
wait(NULL);

return 0;
}
```

3

Codifica el siguiente programa